FlickPick

Members Name/Github:

Matthew Casazza/Casazza2, Rohith Ratheesh/industrybby , Page Howland/page-h, Alitzel Aguilar/alitzel-a, Mason English/Masone1811

Project Description:

FlickPick is a mobile-first web application that reimagines movie discovery by combining the intuitive swipe-based interaction popularized by dating apps with a rich movie database. Users are presented with movie cards featuring posters, ratings, genres, and brief synopses, and can swipe right to add a movie to their watchlist or swipe left to pass. Over time, FlickPick learns each user's preferences and surfaces increasingly personalized recommendations, making it effortless to find your next favorite film.

Beyond individual discovery, FlickPick offers a powerful group-matching feature. Users can create a watch session with friends, and the app identifies movies that everyone in the group has swiped right on — or that align with the group's collective taste. This eliminates the all-too-common struggle of deciding what to watch together, turning a frustrating group decision into a fun, gamified experience.

FlickPick bridges the gap between passive browsing on traditional movie platforms and active, engaging discovery. Whether you're a solo viewer looking for hidden gems or a group of friends planning a movie night, FlickPick provides a fast, fun, and social way to find the perfect film.

Slides

Project Tracker Link:

https://github.com/orgs/CSCI3308-FInal-Project/projects/1/views/1

Video Link:

VCS Link:

https://github.com/CSCI3308-FInal-Project/FlickPick

Contributions:

Page Howland - Developed the Group Sessions feature end-to-end, including the user interface, the front end swipe view, and all the backend routes for creating, joining, swiping, and ending sessions. Added the database tables for group sessions, session members, and session swipes, and notifications to track matches across users in real time. Implemented async and live swiping

modes and added a profile picture upload feature. I also built the registration page confirm password validation component, and created wireframes for the profile, watchlist, and register pages. Lastly, I helped implement the test cases and get Docker Compose working correctly for local development.

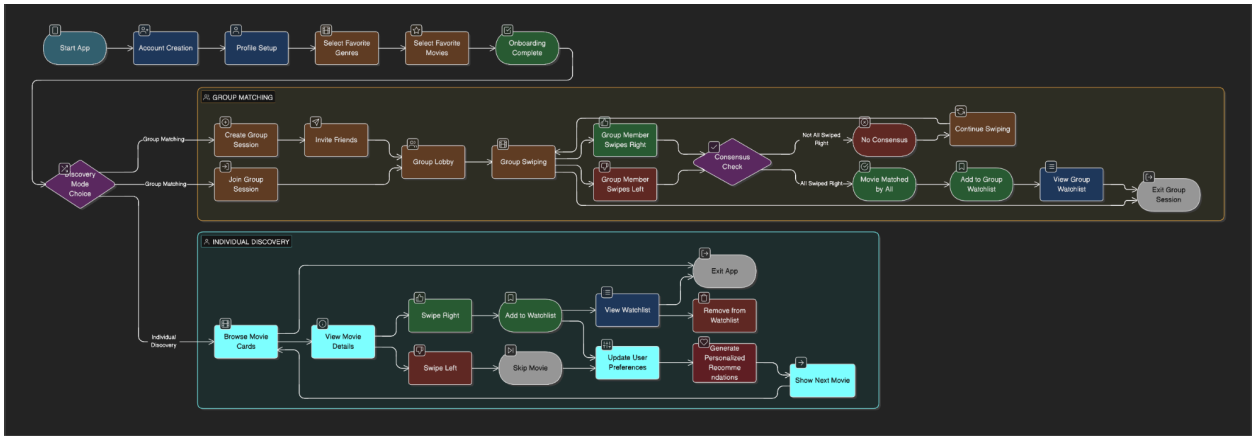
Rohith Ratheesh - Responsible for building and iterating on the core home page experience of FlickPick. I integrated the TMDb API into the home route to dynamically fetch and display real movie data, and built the movie card swipe UI from scratch, including the card layout, swipe animations, and the client side logic to advance cards and POST saves to the watchlist. From there I progressively enhanced the feature: first randomizing the swipe deck by pulling from multiple TMDb categories (popular, top-rated, upcoming, now playing) on random pages so users see a different selection every session, then adding genre and minimum rating filters that route through TMDb's discover endpoint. Finally, I designed and implemented a personalized recommendation algorithm backed by a swipe_history table, recording every pass and save to track each user's genre, actor, and director preferences. The algorithm uses those signals to bias the deck 30% toward the user's top liked genres, 30% toward their most-liked actors and directors via TMDb's people-based discovery, and 40% random variety to keep the experience fresh, only activating once a user has built up enough swipe history.

Matthew Casazza - Contributed 57 commits across the full stack using Node.js, Express, PostgreSQL, Handlebars, and CSS. Key features include: movie reviews, watchlist enhancements (sorting, bulk actions, pagination, watched list), swipe history, and friend activity styles. Also deployed a significant frontend refactor which included cinematic card UI, undo button, sidebar, detail panel, profile page refactoring, keyboard navigation, and genre badge layout.

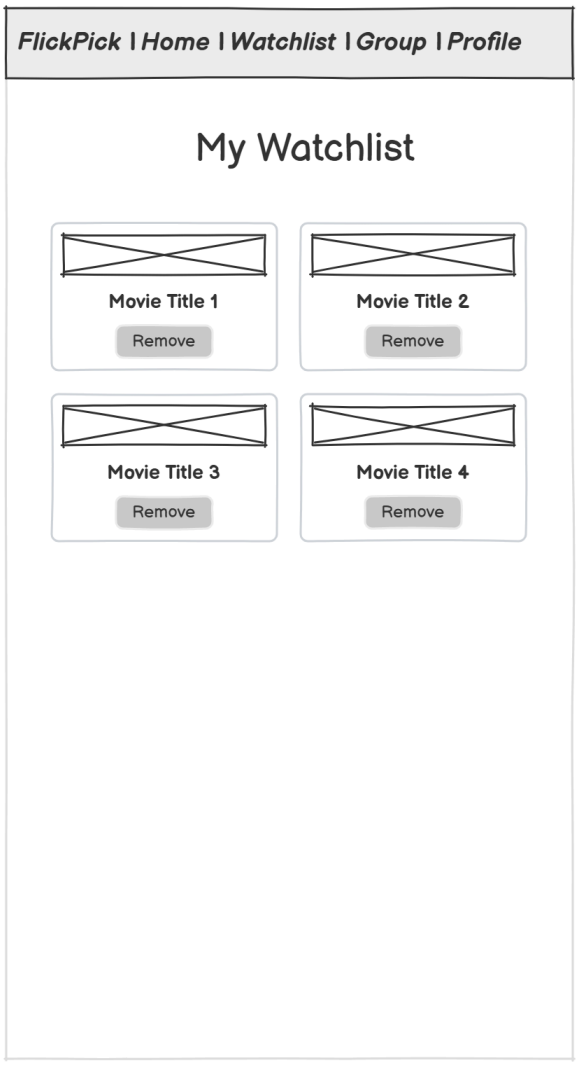
Alitzel Aguilar - I contributed to both frontend and backend development. I enhanced the login system to support email or username authentication and implemented a forgot-password feature. I also improved registration security with a password visibility toggle and real-time strength indicator. Additionally, I developed and debugged the notifications system, wrote test cases for watchlist functionality, and refined CSS styling.

Mason English - I contributed mainly to the front end and backend of the user profile and social features. I designed the profile page, with its sections that can be customized by the user, and tied the sections to the database so they would be persistent. I also worked on the friends system, designing the friends page, and implementing the add friend system using user id keys that are unique to each account. I also combined this work and added the ability to view other users profiles using the persistent profile page data after adding them as a friend.

Use Case Diagram:



Wireframes:



Displays saved movies
The remove button deletes a movie from the watchlist

FlickPick

Register

Username

Email

Password

Confirm Password

Create Account

Already have an account? Log in

Registration Page

Displays error if user input is invalid

Redirects user to login page when registered successfully

FlickPick | Home | Watchlist | Group | Profile

My Profile

Username: [Username]

Favorite Genres

- Action
- Mystery
- Thriller

Swipe History Stats

Total Swipes: 400

Right Swipes: 250

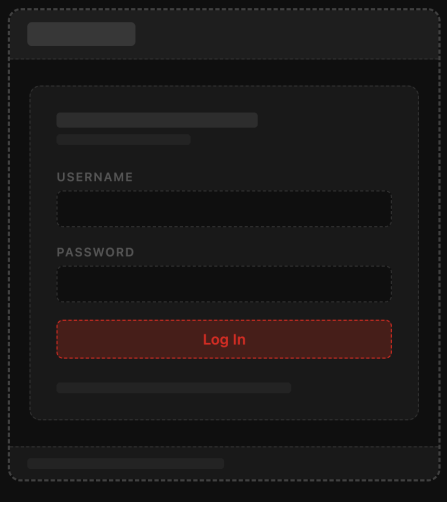
Left Swipes: 150

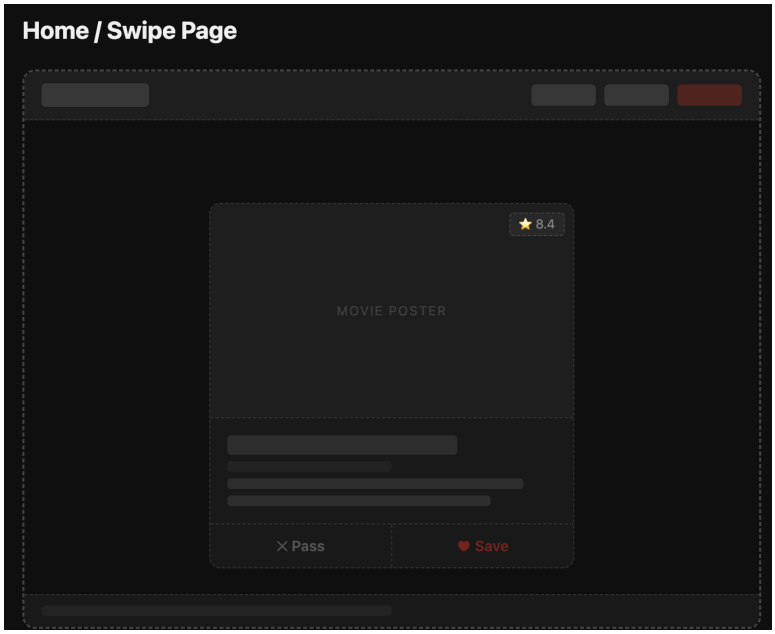
Logout

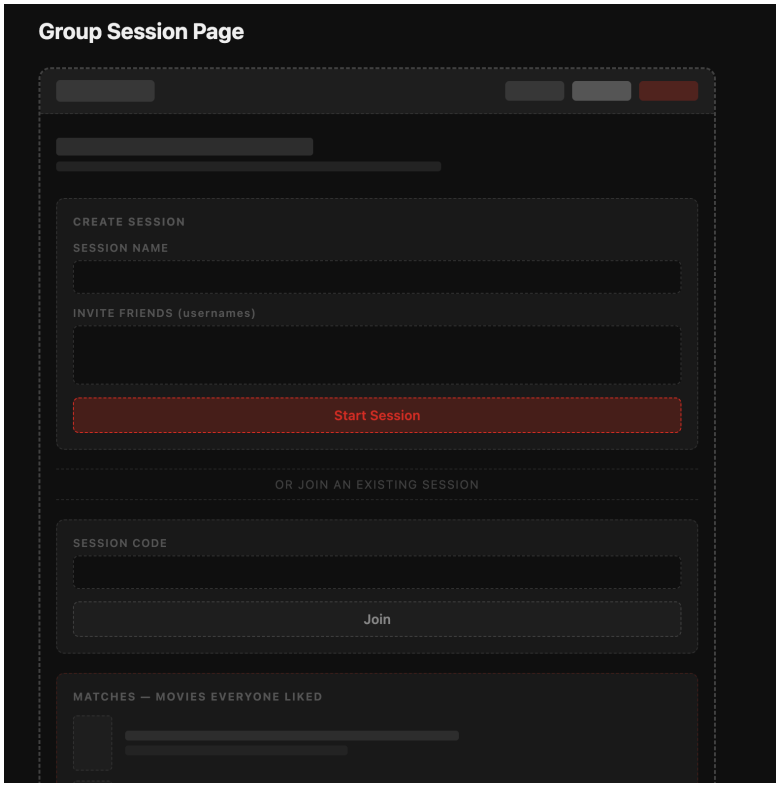
Tracks the user's favorite genres and swipe stats

Logout redirects user to login page

Login Page







Test Result Overview:

Tester 1:

They tested the authentication system by creating an account and logging in. While they understood the password error message, they found it frustrating that feedback only appeared after submission and that they could not see what they were typing, which led to uncertainty about mistakes. Based on this, we implemented a real-time password strength indicator and a

show/hide password toggle to improve clarity, accessibility, and user confidence during registration and login.

Tester 2:
This tester focused on watchlist management features, including saving movies, handling duplicates, and removing items. They reported that the feature worked smoothly and appreciated that the watchlist updated instantly without requiring a page refresh. They especially liked that duplicate movies were automatically prevented from being added, which made the system feel reliable and polished. Their testing confirmed that the watchlist functionality performed as expected and provided positive feedback on the application's responsiveness and ease of use.

UAT Results & Observations:

Feature 1 — User Authentication

TC#	Test Case	Actual Result	Status
TC01	Register with valid credentials	Account created successfully; user redirected to swipe home page as expected.	Pass
TC02	Register with duplicate email	Error message "An account with this email already exists" displayed correctly; no duplicate record created in DB.	Pass
TC03	Register with weak password	Inline error "Password must be at least 8 characters" appeared immediately on submission attempt.	Pass
TC04	Login with valid credentials	Session created; user redirected to swipe home page. Verified via session cookie in DevTools.	Pass
TC05	Login with invalid password	Generic error "Invalid email or password" displayed; no indication of which field was incorrect.	Pass
TC06	Logout	Session cleared on logout; redirect to login page confirmed. Attempting to access a protected route returned 401 as expected.	Pass

Observations: Authentication flows behaved correctly across all tested browsers (Chrome, Firefox, Safari Mobile). Form validations triggered client-side before submission, reducing unnecessary server calls. Session handling was verified to be stateless between tabs, opening a new tab after logout correctly blocked access to protected routes.

Feature 2 — Movie Discovery (Swipe Interface)

TC#	Test Case	Actual Result	Status
TC07	Movie card displays correct info	All cards rendered poster image, title, genre(s), audience rating, and synopsis pulled from TMDb. No missing fields observed across the 50-movie test set.	Pass
TC08	Swipe right saves movie	"Interstellar" added to watchlist on swipe right; next card loaded in under 1 second. Watchlist entry confirmed in DB.	Pass
TC09	Swipe left passes movie	Card dismissed with animation; next card loaded immediately. Confirmed the passed movie was not present in the watchlist table.	Pass
TC10	Previously swiped movies do not reappear	After pre-seeding 20 swiped movies, none reappeared in the feed across an extended browsing session. Query filtering confirmed via DB inspection.	Pass
TC11	Swipe works on mobile touch	Touch swipe gesture registered correctly on both iPhone 14 (Safari) and Android (Chrome). Card slide animation completed smoothly; no gesture drop-offs observed.	Pass
TC12	Filter movies by genre	Selecting "Comedy" genre filter updated the feed to show only comedy titles. Switching back to "All" restored the full unfiltered feed.	Pass

Observations: The swipe feed performed consistently across desktop and mobile form factors. Card load time stayed well within the 1 second threshold in all tested interactions. The genre filter applied without a full page reload, which external testers noted as a smooth experience. No TMDb API errors were encountered during the test window using the development key.

Feature 3 — Watchlist Management

TC#	Test Case	Actual Result	Status
TC13	View saved movies in watchlist	All 5 pre-seeded movies displayed correctly as grid cards with poster, title, and genre. Layout rendered properly on both desktop and mobile viewports.	Pass
TC14	Remove a movie from watchlist	"The Dark Knight" removed after confirmation prompt; list updated dynamically without a full page reload. Removal confirmed in DB.	Pass

TC15	Empty watchlist shows helpful message	Fresh account with 0 saved movies correctly displayed the empty state message: "No movies saved yet. Start swiping!"	Pass
TC16	Duplicate movies not added to watchlist	Swiping right a second time on a movie already in the watchlist produced no duplicate entry; watchlist count remained the same. Silently ignored as specified.	Pass
TC17	Watchlist sorted by most recently saved	Three movies saved at distinct times appeared in correct descending order (most recent first). Sort order verified against DB timestamps.	Pass

Observations: Watchlist management was the most user-facing feature tested by the external tester, who found the confirmation prompt on removal to be intuitive and appropriately cautious. The dynamic removal (no full reload) was noted positively. Duplicate prevention worked transparently, the user received no disruptive error, which matches the "silently ignored" spec. Sort order held correctly even after removing and re-adding a movie.

Summary

All 17 test cases across 3 features passed with no failures. No GitHub issues were filed under the UAT-bug tag. The application is confirmed to meet all acceptance criteria.

Deployment:
https://flickpick-9e0f.onrender.com/

Extra Credit

1. Innovative Problem Selection
    i. Flickpick is meant to address the problem that many groups of friends find themselves facing: Picking a movie to watch. The target audience is social groups of people that want to find a movie to watch that everyone will be interested in.
    ii. Our group discussed amongst ourselves what problems that we have all found ourselves in, and one of those problems was that we all had trouble picking something to watch when spending time with our friends. So we knew that we all had that problem, and so did our friends, and most likely, their friends too.

iii. Existing movie platforms simply rely on basic recommendations based on the person that usually uses their app. Meaning, that there will not be special recommendations for group watch parties. FlickPick pixes this by generating recommendations based on multiple people and identifying which movies everyone will be interested in watching,
iv. FlickPick goes beyond basic tutorial style apps by integrating the TMDB api, swipe actions, personalized recommendations, and group social features.
v. We performed user acceptance testing with authentication, movie discovery, and watchlist features. Feedback from one test led to adding a password strength indicator and a hide/reveal feature.

2. Creativity of the Website from the UI/UX perspective
    i. FlickPick uses a consistent color scheme and a similar segmented design across all major pages. The focus is on the movies and the app design feels unified.
    ii. The navigation of the website is simple, you are presented with movies to swipe on as soon as you log in, and it is simple to reach the other pages as they are clearly labeled across the top menu.
    iii. FlickPick uses custom styling on every page following a similar theme. Card layouts, grids, friend pages, dropdown menus, etc., all show more UI work than minor CSS tweaks.
    iv. The app includes swipe animations, notification alerts, watchlist updates, personal customization and more. These elements make the interface interactive and responsive.
    v. The colors were intentionally chosen to make sure that all text is readable and clear, it was also intentional to make the main swiping features large so it is easy to operate. The show/hide password feature was also added because of these considerations.